Faculty of computers and data science
Data Structures
Spring 2026

LRU Cache System

Team members : 10 **Deadline: 9/5/2026**
Team Formation will be open in **4/5/2026** :
Data Structures final project – Click to fill out form

Project overview

In this project, you will implement an **LRU (Least Recently Used) Cache System** that supports fast data access and automatic eviction of outdated items.

Modern systems such as web browsers, operating systems, and databases use caching to improve performance. However, cache size is limited, so systems must decide: **Which item should be removed when the cache is full?**

This project solves that problem using the **LRU policy**, where the **least recently used item is evicted first**.

System Requirements

Your LRU Cache must support the following operations:

Operation	Description	Required Complexity
get(key)	Retrieve value if exists	O(1)
put(key, value)	Insert/update value	O(1)
remove(key)	Remove specific key	O(1)
display()	Show cache state (most → least recent)	O(n)

Core Design Constraint

You are **NOT** allowed to use built-in data structures like:

- LinkedHashMap
- HashMap (for full solution logic)

You must implement:

- **Hash Table from scratch**
- **Doubly Linked List from scratch**

Faculty of computers and data science
Data Structures
Spring 2026

LRU Cache System

System Design

Why Two Data Structures?

Structure	Role
Hash Table	Direct access by key → O(1)
Doubly Linked List	Maintain usage order

High Level Architecture

- Hash Table → maps key → node
- Doubly Linked List → maintains order:
  - **Head = Most Recently Used**
  - **Tail = Least Recently Used**

Implementation Tasks

Task 1 Node Design

Design the basic unit of the system.

Create a **Node** class that represents an element in the cache.
Each node must contain:

- key
- value
- reference to the previous node
- reference to the next node

This node will be shared between the hash table and the linked list.

LRU Cache System

Task 2 Doubly Linked List Initialization

Build the structure that maintains the order of usage.

Implement a doubly linked list with:

- a head pointer representing the most recently used item
- a tail pointer representing the least recently used item

Ensure the list handles the empty case correctly.

Task 3 Insert at Front

Add a node to the beginning of the list.

Implement a method to insert a node at the front.
This operation marks an item as recently used.

Handle:

- updating the head
- inserting into an empty list

Task 4 Remove a Node

Remove a node from any position in the list.

Implement a method that removes a given node.
Ensure correct pointer updates in all cases:

- removing the head
- removing the tail
- removing a middle node

LRU Cache System

Task 5 Move Node to Front

Update the position of a node after access.

Implement a method that moves a node to the front of the list.
This reflects that the item has been recently used.

The operation should:

- remove the node from its current position
- insert it again at the front

Task 6 Remove Least Recently Used Item

Support the eviction policy.

Implement a method to remove the node at the tail of the list.
This node represents the least recently used item.

Return the removed node so it can also be deleted from the hash table.

Task 7 Hash Function and Table Setup

Prepare fast access to cache elements.

Design a hash function that maps keys to indices.
Implement a hash table that stores:

- key mapped to node reference

Ensure proper handling of collisions.

LRU Cache System

Task 8 Hash Table Operations

Implement the core operations of the hash table.

Provide methods to:

- insert a key and node
- search for a key
- remove a key

All operations should aim for constant time on average.

Task 9 LRU Cache Core Logic

Build the main cache system.

Implement the cache class that combines the hash table and the linked list.

Required operations:

- get(key)
  - return the value if it exists
  - move the node to the front
- put(key, value)
  - insert a new item or update an existing one
  - move the node to the front

This task ensures coordination between both structures.

LRU Cache System

Task 10 Capacity Handling and Testing

Complete the system and verify correctness.

First, implement the capacity rule:

- if the cache is full:
  - remove the least recently used node from the list
  - remove its key from the hash table

Then create test cases to verify:

- correct insertion and retrieval
- correct update behavior
- correct eviction order
- handling of edge cases such as empty cache or repeated access

Testing Requirements

Students must test:

- Insert until full capacity
- Access elements repeatedly
- Eviction correctness
- Updating existing keys
- Removing elements

Deliverables

1. Source Code : Clean, modular Java implementation

2. Report

Must include:

- **System design explanation**
- **Complexity analysis**
- **Test cases**

Faculty of computers and data science
Data Structures
Spring 2026

LRU Cache System

Complexity Analysis

Operation	Time
get	O(1)
put	O(1)
remove	O(1)
eviction	O(1)

Grading Rubric

Criteria	Weight
Correctness	30%
Data Structure Implementation	25%
Design & Integration	20%
Complexity Analysis	10%
Code Quality	10%
Discussion	5%